SPRING JPA + OAUTH + JWT

— IN-DETAIL HANDBOOK (ADVANCED LEVEL) —

Aman_views

---

1. INTRODUCTION

This handbook covers Spring Data JPA in depth followed by secure authentication & authorization using JWT and OAuth 2.0. Together, these technologies are widely used to build robust, scalable and secure REST APIs in modern enterprise applications.

2. WHAT IS JPA ?

* JPA (Java Persistence API) is a specification for managing and persisting data between Java objects and relational databases.
* It is a standard defined by Java EE (Jakarta EE).
* JPA does not provide the implementation. Hibernate is the most popular JPA implementation.
* JPA works on ORM (Object Relational Mapping) concept.

3. JPA vs HIBERNATE vs JDBC

* JDBC : Low level API, lots of boilerplate code, manual resource management, no ORM.
* Hibernate : Full ORM framework, provides extra features (caching, lazy loading, etc.).
* JPA : Specification / Standard only (API). Implemented by providers like Hibernate.

4. WHY SPRING DATA JPA ?

* Simplifies data access layer development.
* Provides ready-made repository interfaces.
* Supports custom queries, pagination, sorting easily.
* Integrates smoothly with Spring ecosystem.
* Reduces boilerplate code significantly.

SPRING JPA + OAUTH + JWT

— IN-DETAIL HANDBOOK (ADVANCED LEVEL) —

Aman_views

5. JPA ARCHITECTURE (WORKING FLOW)

* Controller receives request from client.
* Service layer contains business logic.
* Repository (extends JpaRepository) interacts with JPA.
* JPA uses EntityManager to perform operations.
* Hibernate (JPA Provider) generates SQL and communicates with Database.

KEY COMPONENTS :

1.	Entity	:	Represents table (Java class mapped to DB table).
2.	EntityManager	:	Interface used to perform CRUD operations.
3.	Persistence Context	:	First level cache, manages entity lifecycle.
4.	Repository	:	Provides abstraction over EntityManager.
5.	JPA Provider	:	Hibernate (implementation of JPA).

6. ENTITY EXAMPLE

```
@Entity                                          // Marks this class as JPA Entity
@Table(name = "users")                           // Maps to table name
public class User {
    @Id                                                          // Primary key
    @GeneratedValue(strategy = GenerationType.IDENTITY)          // Auto increment
    private Long id;

    @Column(name = "name", nullable = false, length = 100)       // Column mapping
    private String name;

    @Column(unique = true, nullable = false)                     // Unique email
    private String email;
    @Enumerated(EnumType.STRING)                                 // Enum as String
    private Role role;
    @CreationTimestamp                                           // Auto set on insert
    private LocalDateTime createdAt;
    @UpdateTimestamp                                             // Auto set on update
    private LocalDateTime updatedAt;
    @Transient                                                   // Not persisted in DB
    private String tempData;
}
```

NOTE :
* JPA works as a specification.
* Hibernate is the default implementation used by Spring Boot.
* Always use Repository layer instead of EntityManager directly.
* Prefer annotations over XML configuration.

SPRING JPA + OAUTH + JWT

— IN-DETAIL HANDBOOK (ADVANCED LEVEL) —

Aman_views

7. REPOSITORY LAYER & QUERY METHODS

Repository is an interface that provides CRUD operations and supports custom queries.

A. REPOSITORY INTERFACES (Provided by Spring Data JPA)

* JpaRepository <T, ID> → Full CRUD + Pagination + Sorting
* CrudRepository <T, ID> → Basic CRUD operations
* PagingAndSortingRepository <T, ID> → Pagination + Sorting
* JpaSpecificationExecutor <T> → Specifications, Criteria queries

METHODS PROVIDED

save()
findById()
findAll()
deleteById()
count()
existsById()
...

B. DERIVED QUERY METHODS

Spring Data JPA creates queries automatically from method names.

EXAMPLES

findByEmail(String email)	Find by email
findByName(String name)	Find by name
findByAgeGreaterThan(int age)	Find users with age > given age
findByNameContaining(String name)	Find name containing given text
findByStatusOrderByCreatedAtDesc(String status)	Find by status and sort by createdAt desc

C. @QUERY ANNOTATION

Custom JPQL queries using @Query.

```
public interface UserRepository extends JpaRepository<User, Long> {

 @Query("SELECT u FROM User u WHERE u.email = :email")
 User findUserByEmail(@Param("email") String email);

}
```

D. NATIVE QUERY

Execute native SQL queries.

```
public interface UserRepository extends JpaRepository<User, Long> {

  @Query(value = "SELECT * FROM users WHERE status = ?1",
                 nativeQuery = true)
    List<User> findByStatus(String status);
}
```

E. PAGINATION

Fetch large data in pages.

```
Page<User> findByStatus(String status, Pageable pageable);
PageRequest page = PageRequest.of(0, 10, Sort.by("name").asc());
Page<User> result = repo.findByStatus("ACTIVE", page);
```

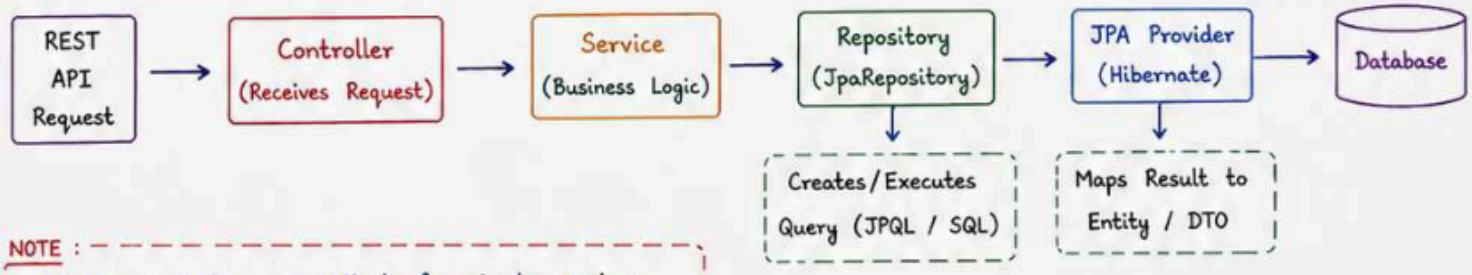
F. SORTING

Sort results by one or multiple fields.

```
Sort sort = Sort.by(Sort.Direction.DESC, "createdAt");

List<User> list = repo.findAll(sort);
```

G. REPOSITORY WORKING FLOW



NOTE :

* Prefer derived query methods for simple queries.
* Use @Query for complex queries.
* Use Pagination & Sorting to handle large datasets.
* Always use DTO Projection for performance optimization when needed.

SPRING JPA + OAUTH + JWT

— IN-DETAIL HANDBOOK (ADVANCED LEVEL) —

Aman_views

4. ENTITY RELATIONSHIPS

Entity Relationships define how one entity is connected with another in the database. Spring Data JPA provides annotations to map these relationships automatically.

1. @OneToOne

One record ↔ One record

Example : User ↔ Passport
Employee ↔ Locker

```
@OneToOne
@JoinColumn(name = "passport_id")
private Passport passport;
```

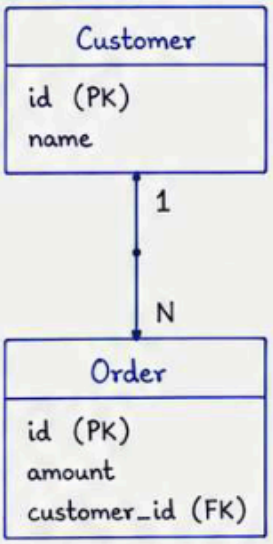
2. @OneToMany

One parent can have multiple child records.

Example : Customer → Orders
Department → Employees

```
@OneToMany(mappedBy = "customer",
           cascade = CascadeType.ALL)
private List<Order> orders;
```

RELATIONSHIP DIAGRAM



One Customer can have Multiple Orders.

3. @ManyToOne

Many child records belong to one parent.

```
@ManyToOne
@JoinColumn(name = "customer_id")
private Customer customer;
```

4. @ManyToMany

Many records ↔ Many records

Example : Student ↔ Course
User ↔ Role

```
@ManyToMany
@JoinTable(
      name = "student_course")
)
private List<Course> courses;
```

IMPORTANT ANNOTATIONS

* mappedBy → Defines owner side
* @JoinColumn → Creates Foreign Key
* @JoinTable → Creates Mapping Table
* cascade → Propagates operations
* fetch → Controls loading strategy

FETCH TYPES

LAZY
- Loads only when required
- Better Performance
- Recommended

EAGER
- Loads immediately
- Can reduce performance

CASCADE TYPES

- PERSIST
- MERGE
- REMOVE
- REFRESH
- DETACH
- ALL

NOTE : Prefer LAZY Fetch and use CascadeType.ALL only when the child entity completely depends on the parent.

SPRING JPA + OAUTH + JWT

— IN-DETAIL HANDBOOK (ADVANCED LEVEL) —

Aman_views

5. ADVANCED JPA CONCEPTS

Advanced JPA features help in building high performance, scalable and maintainable applications.

1. PAGINATION

- Used to fetch large data in small chunks.
- Improves performance and memory usage.

```
Pageable pageable = PageRequest.of(0, 10, Sort.by("id").dssc());
Page<User> page = userRepository.findAll(pageable);
List<User> users = page.getContent();
```

2. SORTING

- Sort results by one or multiple fields.

```
Sort sort = Sort.by(Sort.Direction.ASC, "name");
List<User> list = userRepository.findAll(sort);
```

3. SPECIFICATIONS (JPA SPEC)

- Build dynamic queries using Specification interface.
- Useful for advanced search with multiple filters.

```
Specification<User> hasName = (root, query, cb) ->
    cb.equal(root.get("name"), "John");
Specification<User> hasAge = (root, query, cb) ->
    cb.greaterThan(root.get("age"), 18);
Specification<User> spec = hasName.and(hasAge);
List<User> list = userRepository.findAll(spec);
```

4. CRITERIA API

- Programmatic way to build type-safe queries.
- Alternative to JPQL.

```
CriteriaBuilder cb = entityManager.getCriteriaBuilder();
CriteriaQuery<User> cq = cb.createQuery(User.class);
Root<User> root = cq.from(User.class);
Prodicate p = cb.equal(root.get("status"), "ACTIVE");
cq.select(root).where(p);
List<User> results = entityManager.createQuery(cq).getResultList();
```

5. PROJECTIONS

- Fetch only required fields instead of entire entity.
- Improves performance.

a) Interface Based Projection

```
public interface UserNameView {
    String getName();
    String getEmail();
}
```

```
List<UserNameView> list = userRepository.findByStatus("ACTIVE");
```

b) DTO Projection (JPQL Constructor Expression)

```
@Query("SELECT new com.app.dto.UserDTO(u.id, u.name) " +
       "FROM User u WHERE u.status = :status")
List<UserDTO> list = findByStatus(Status status);
```

6. TRANSACTIONS

- Ensures data consistency.
- @Transactional provides declarative transaction management.

```
@Transactional
public void placeOrder(Long userId, Long productId) {
    // multiple DB operations
}
```

```
Default Propagation : REQUIRED
Default Isolation   : DEFAULT
Default ReadOnly    : false
```

7. LAZY vs EAGER LOADING

Feature	LAZY	EAGER
Loading	Loads only when needed	Loads immediately
Performance	Better (recommended)	Can cause performance issues
Use Case	Most of the cases	When data is always required
Annotation	fetch = FetchType.LAZY	fetch = FetchType.EAGER

8. N+1 SELECT PROBLEM

- Occurs when we fetch parent entities and then access child entities lazily in a loop.
- Solution: Use JOIN FETCH or @EntityGraph.

Example:

```
@Query("SELECT u FROM User u JOIN FETCH u.orders WHERE u.id = :id")
User findUserWithOrders(@Param("id") Long id);
// or
@EntityGraph(attributePaths = {"orders"})
Optional<User> findById(Long id);
```

9. OPTIMISTIC LOCKING

- Prevents data inconsistency in concurrent updates.
- Uses @Version field.

```
@Version
private Long version;
```

NOTE :
- ✓ Use Pagination & Projection to improve performance.
- ✓ Prefer LAZY loading to avoid unnecessary data fetch.
- ✓ Use @Transactional at Service layer.
- ✓ Monitor and handle N+1 problem in production.

SPRING JPA + OAUTH + JWT

— IN-DETAIL HANDBOOK (ADVANCED LEVEL) —

Aman_views

6. TRANSACTIONS & CONCURRENCY (JPA)

Transactions ensure data consistency and integrity. JPA works with Spring's @Transactional annotation.

1. @TRANSACTIONAL

* Makes method or class transactional.
* If an error occurs, changes are rolled back.
* Default propagation = REQUIRED.

```
@Service
public class OrderService {
    @Transactional
    public void placeOrder(Long userId, Long productId) {
        // multiple DB operations
    }
}
```

2. PROPAGATION TYPES

* REQUIRED : Joins existing tx or creates new one.
* REQUIRES_NEW : Suspends current tx and creates new one.
* SUPPORTS : Runs in tx if exists, otherwise non-transactional.
* NOT_SUPPORTED : Executes non-transactionally.
* MANDATORY : Must run in a tx or throws exception.
* NEVER : Must run outside a tx.
* NESTED : Executes within nested tx (uses savepoints).

Default Propagation for @Transactional is REQUIRED.

3. ISOLATION LEVELS

Isolation Level	Description
DEFAULT	Uses database default isolation level.
READ_UNCOMMITTED	Can read uncommitted changes. (Dirty Read)
READ_COMMITTED	Reads committed data only.
REPEATABLE_READ	Same data will be read again and again.
SERIALIZABLE	Highest level. Completely isolated.

Higher isolation = More safety but lower performance.

4. LOCKING IN JPA

a) PESSIMISTIC LOCK

* Locks the data at database level.
* Other transactions cannot modify locked data.
* Use cases: Banking, Order processing.

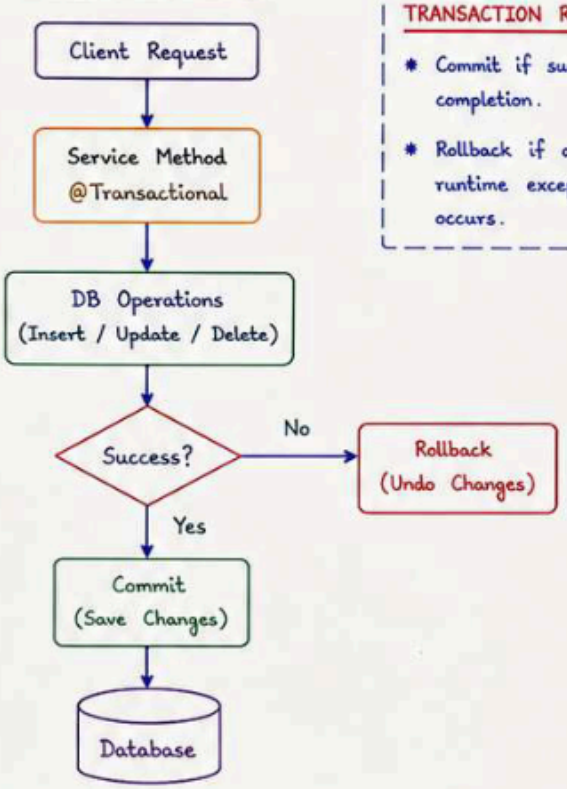
```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT o FROM Order o WHERE o.id = :id")
Order findByIdForUpdate(@Param("id") Long id);
```

b) OPTIMISTIC LOCK

* Does not lock data.
* Uses @Version field to check data modification.
* If data is changed by another tx, OptimisticLockException is thrown.

```
@Entity
public class Product {
    @Id @GeneratedValue
    private Long id;
    private String name;
    @Version
    private Long version;
}
```

5. TRANSACTION FLOW



TRANSACTION RULE

* Commit if successful completion.
* Rollback if any runtime exception occurs.

6. READ ONLY TRANSACTION

* Optimizes performance for read-only operations.
* No dirty checking, no write operations.

```
@Transactional(readOnly = true)
public List<User> getAllUsers() {
    return userRepository.findAll();
}
```

7. TIMEOUT

* Used to set maximum time a transaction can run.
* If exceeds, transaction is rolled back.

```
@Transactional(timeout = 10)
public void callExternalService() {
    // long running operation
}
```

8. BEST PRACTICES

* Keep transactions short and simple.
* Use readOnly = true for read operations.
* Avoid unnecessary locks.
* Handle exceptions properly, let Spring rollback.
* Use proper isolation level based on use case.
* Monitor long running transactions.

NOTE :
- ✓ Use @Transactional at Service layer.
- ✓ Avoid placing @Transactional on private methods.
- ✓ Choose Locking strategy based on use case.
- ✓ Prefer Optimistic Locking for better performance.

SPRING JPA + OAUTH + JWT

— IN-DETAIL HANDBOOK (ADVANCED LEVEL) —

Aman_views

6. JWT AUTHENTICATION

JWT (JSON Web Token) is a compact, URL-safe token used to securely transmit information between parties as a JSON object.

1. WHAT IS JWT?

* Stateless authentication mechanism.
* Token is signed using a secret key.
* Client stores token and sends with each request.
* Server validates token and grants access.

2. JWT STRUCTURE



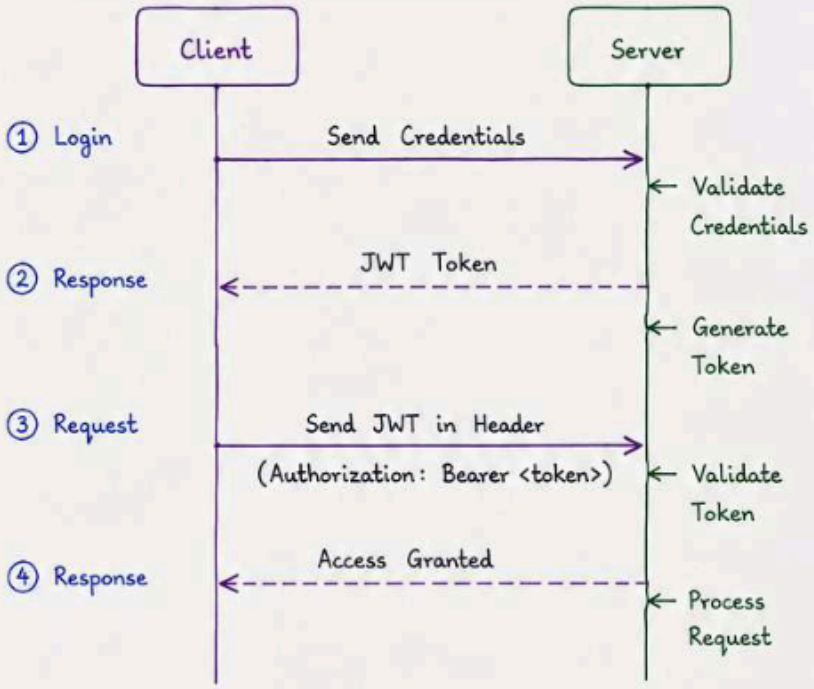
Example :

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpZCI6MSwicm9sZSI6IlVJRVIiLCJleHAiOjE3MTU2MDAwMDB9.
QWERTYUIOPASDFGHJKLZXCVBNM1234567890

3. JWT COMPONENTS

- **Header**
  - Contains type of token and signing algorithm.
- **Payload**
  - Contains claims (data).
  - Standard claims: sub, exp, iat, iss, etc.
- **Signature**
  - Created using header, payload and secret key.
  - Ensures token is not tampered.

4. JWT AUTHENTICATION FLOW



5. WHERE IS JWT STORED?

* LocalStorage (Web)
* SessionStorage (Web)
* HttpOnly Cookie (Recommended)
* Mobile Secure Storage

6. ADVANTAGES

- ✓ Stateless
- ✓ Scalable
- ✓ Cross-domain
- ✓ Compact
- ✓ Can be used for Authorization (Scopes / Roles)

7. DISADVANTAGES

- X Cannot be revoked easily
- X Token size increases with more claims
- X Sensitive data should not be stored in payload
- X Short expiry + Refresh Token management is required

8. JWT BEST PRACTICES

* Always use HTTPS
* Keep token short-lived
* Store minimal data in payload
* Use strong secret key
* Blacklisting / Refresh Token for logout

NOTE :
* JWT is not encrypted, it is only signed.
* Never store sensitive information (password, card details, etc.) in JWT.
* Use Refresh Token for long sessions.

SPRING JPA + OAUTH + JWT

— IN-DETAIL HANDBOOK (ADVANCED LEVEL) —

Aman_views

7. JWT IMPLEMENTATION

1) Dependencies (Maven)

```
<dependency>
   <groupId>io.jsonwebtoken</groupId>
   <artifactId>jjwt-api</artifactId>
   <version>0.11.5</version>
</dependency>
// Add jjwt-impl and jjwt-jackson
   for runtime
```

2) JWT Utility Class

```
@Component
public class JwtUtil {
    private String secret = "MySecretKey@1234567890";
    private long expirationMs = 1000 * 60 * 60; // 1 hour

    public String generateToken(String username, List<String> roles) {
        return Jwts.builder()
            .setSubject(username)
            .claim("roles", roles)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + expirationMs))
            .signWith(SignatureAlgorithm.HS256, secret)
            .compact();
    }
    public Claims validateToken(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(secret)
            .build()
            .parseClaimsJws(token)
            .getBody();
    }
}
```

3) AuthController (Login)

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

  @PostMapping("/login")
  public ResponseEntity<?> login(@RequestBody LoginRequest req) {
     // 1. Authenticate user (from DB)
     User user = userService.authenticate(req.getUsername(),
                         req.getPassword());
     // 2. Generate Token
     String token = jwtUtil.generateToken(
           user.getUsername(), user.getRoles());

     // 3. Return Token
     return ResponseEntity.ok(new JwtResponse(token));
  }
}
```

4) JWT Filter (Validate Token)

```
public class JwtFilter extends OncePerRequestFilter {
  @Override
  protected void doFilterInternal(HttpServletRequest req,
      HttpServletResponse res, FilterChain chain)
      throws ServletException, IOException {
      String authHeader = req.getHeader("Authorization");
      if (authHeader != null && authHeader.startsWith("Bearer ")) {
         String token = authHeader.substring(7);
         Claims claims = jwtUtil.validateToken(token);
         String username = claims.getSubject();
         // Set authentication in context
         UsernamePasswordAuthenticationToken auth =
             new UsernamePasswordAuthenticationToken(
                 username, null, getAuthorities(claims));
         SecurityContextHolder.getContext().setAuthentication(auth);
  }
  chain.doFilter(req, res);
}
```

5) Add Filter in Security Config

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
  http
    .csrf(csrf -> csrf.disable())
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/auth/**").permitAll()
        .anyRequest().authenticated())
    .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
  return http.build();
}
```

6) Response DTO

```
public class JwtResponse {
    private String token;
    public JwtResponse(String token) {
        this.token = token;
    }
    public String getToken() {
        return token;
    }
}
```

FLOW DIAGRAM



NOTE :
- Keep secret key strong and secure.
- Token is stateless, so server does not store session.
- Use HTTPS in production.
- Use Refresh Token for long sessions.

JWT IN HEADER

```
Authorization: Bearer <your_token>
Example:
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....
```

SPRING JPA + OAUTH + JWT

— IN-DETAIL HANDBOOK (ADVANCED LEVEL) —

Aman_views

1. WHAT IS OAUTH2?

- OAuth2 is an authorization framework.
- It allows third-party applications to access resources on behalf of a user.
- Uses Access Token and Refresh Token.

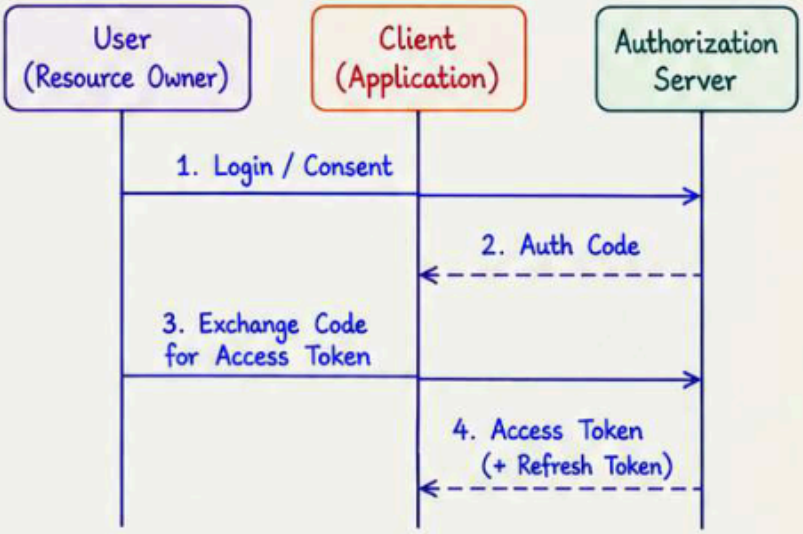
2. KEY TERMS

- Resource Owner : User who owns the data.
- Client : Application requesting access.
- Authorization Server : Verifies user and issues tokens.
- Resource Server : Hosts protected resources (APIs).

3. OAUTH2 GRANT TYPES

Grant Type	Use Case
Authorization Code	Web / Mobile apps (most secure)
Client Credentials	Service to service (no user)
Implicit	Browser-based apps (legacy)
Resource Owner Password	Trusted apps (not recommended)

4. OAUTH2 FLOW (AUTHORIZATION CODE)



5. ACCESS TOKEN vs REFRESH TOKEN

Token	Purpose
Access Token	Short-lived token used for API access.
Refresh Token	Long-lived token used to get new access tokens.

6. OAUTH2 WITH SPRING SECURITY

- Use Spring Security OAuth2 support.
- Configure with client id, client secret and authorization server details.
- Add @EnableWebSecurity and OAuth2 filters.
- Use Resource Server to validate tokens.

7. OAUTH2 ENDPOINTS

/oauth/authorize	→	User authorization
/oauth/token	→	Token issuance
/oauth/check_token	→	Token validation
/oauth/revoke	→	Token revocation

8. COMMON USE CASES

- Login with Google / GitHub.
- Third-party app access (e.g., mobile app).
- Microservices authentication.
- API access with limited scope.

NOTE :

- OAuth2 is more secure than simple username/password.
- Use Authorization Code grant for web applications.
- Always validate tokens on Resource Server.
- Combine with JWT for stateless authentication.

SPRING JPA + OAUTH + JWT

— IN-DETAIL HANDBOOK (ADVANCED LEVEL) —

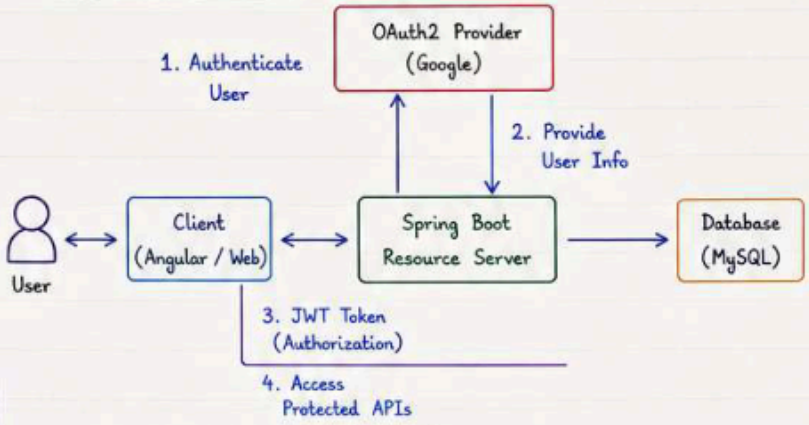
Aman_views

9. OAUTH2 WITH SPRING SECURITY

WHAT WE WILL BUILD

We will secure our Spring Boot application using OAuth2 login (Google) and issue JWT token for API access. ✓

1. ARCHITECTURE OVERVIEW



2. FLOW

1. User clicks "Login with Google".
2. Redirect to Google OAuth2 consent screen.
3. Google authenticates user and sends authorization code to our app.
4. Our app exchanges code for access token (using Client ID & Secret).
5. We get user info from Google.
6. If user is new → create user in DB.
7. Generate JWT token and send to client.
8. Client uses JWT token to access APIs.

3. DEPENDENCIES (MAVEN)

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.3</version>
</dependency>
```

4. APPLICATION.PROPERTIES

```properties
spring.security.oauth2.client.registration.google.client-id= YOUR_CLIENT_ID
spring.security.oauth2.client.registration.google.client-secret= YOUR_SECRET
spring.security.oauth2.client.registration.google.scope= email,profile

app.jwt.secret= MySecretKey@1234567890!  # use strong secret
app.jwt.expiration= 86400000  # 1 day in ms
```

5. SECURITY CONFIGURATION

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

  @Bean
  public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
      .csrf(csrf -> csrf.disable())
      .authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/auth/**").permitAll()
        .anyRequest().authenticated()
      )
      .oauth2Login(oauth -> oauth.defaultSuccessUrl("/login/success", true))
      .oauth2ResourceServer(oauth -> oauth.jwt());

    return http.build();
  }
}
```

6. OAUTH2 LOGIN SUCCESS HANDLER

```java
@Component
public class OAuth2SuccessHandler implements AuthenticationSuccessHandler {
  @Autowired
  private JwtUtil jwtUtil;

  @Override
  public void onAuthenticationSuccess(HttpServletRequest request,
    HttpServletResponse response, Authentication authentication)
    throws IOException {

    String token = jwtUtil.generateToken(authentication);
    response.sendRedirect("http://localhost:4200/login/success?token=" + token);
  }
}
```

7. JWT UTILITY (GENERATE TOKEN)

```java
public String generateToken(Authentication authentication) {
  String username = authentication.getName();
  return Jwts.builder()
    .setSubject(username)
    .setIssuedAt(new Date())
    .setExpiration(new Date(System.currentTimeMillis() + expiration))
    .signWith(SignatureAlgorithm.HS256, secretKey)
    .compact();
}
```

8. PROTECTED API EXAMPLE

```java
@GetMapping("/api/hello")
public ResponseEntity<String> hello() {
  return ResponseEntity.ok("Hello, You are authenticated ✓");
}
```

9. TESTING FLOW

1. Click "Login with Google"
2. Get redirected and login with Google account
3. Backend returns JWT token
4. Use token in Authorization header:
   `Authorization: Bearer <token>`
5. Access protected APIs ✓

NOTE :
- OAuth2 for Authentication (Login)
- JWT for Authorization (Access APIs)
- Spring Security acts as a bridge.

OAuth2 (Login) + JWT (API Access) = SECURE APPLICATION